

USB 8-digit Display

User Manual

by Pedro Reis and João Maia

Table of Contents

1 Introduction.....	3
2 Installation.....	4
3 Software Usage.....	5
3.1 Introduction.....	5
3.2 Requirements.....	5
3.3 Recommendations.....	5
3.4 The API.....	5
3.4.1 Useful info for the correct API usage.....	5
3.4.2 Jackpot Display access API.....	7
3.4.2.1 Write data on the display.....	7
3.4.2.2 Special effects.....	8
3.4.2.3 Fonts.....	9
3.4.2.4 Generic commands.....	9
3.5 Demo program.....	10
4 Hardware.....	14
4.1 Device Description.....	14
4.2 Technical Specifications.....	14
4.3 Electrical Schematic.....	15
4.4 Board Layout.....	16
4.4.1 Top View.....	16
4.4.2 Bottom View.....	17
5 FAQ.....	18

1 Introduction

This is an USB board for 8 digit displays. It was developed and programmed for showing the jackpot value in casino gaming machines. It may also be used to present any number with decimal point, words or animations. It was tested to operate in Windows and Linux OS.

This manual gives a detailed description on how to use the device.

The system and hardware responsible was Pedro Reis and the software development was done by João Maia.

2 Installation

1. Connect the display to the device. This device was projected to be used with SunLED DUR57C-A and XDMR38C based displays.
2. Connect 12V to pin 4 of J2 interface (page 16) and ground to pin 2 and/or 3.
3. If the Operating System is Linux, [libusb 1.0](http://www.libusb.org) (www.libusb.org) must be installed.
4. Connect the device to the machine via a mini USB type B cable.
5. The drivers are automatically installed.
6. If in Windows, run the program that must have the [UsbHidApi](http://www.kadtronix.com/usbhidapi.htm) (<http://www.kadtronix.com/usbhidapi.htm>) library.

If in Linux, run the program.

3 Software Usage

3.1 Introduction

This document gives an indication on the usage of the Gaming Jackpot Display. The Application Programming Interface (API) has been mainly designed to be run on Linux. Keep in mind that the communication with the display is unidirectional, meaning that no data can be read from the device.

3.2 Requirements

libusb-1.0 – this library can be found at www.libusb.org

3.3 Recommendations

We recommend where possible you design and develop your application using the information stated in this document.

3.4 The API

3.4.1 Useful info for the correct API usage

This chapter is devoted to some explanations that could help to develop applications which use the display.

Available constant types and structures:

Intensity

```
enum E_Intensity
{
    E_Intensity_PREV = 0,
    E_Intensity_1,
    E_Intensity_2,
    E_Intensity_3,
    E_Intensity_4
};
```

E_Intensity_PREV is to be used to keep the current display intensity. Values from *E_Intensity_1* to *E_Intensity_4* represent the four levels of intensity being *E_Intensity_1* the darkest and *E_Intensity_4* the brightest.

FX

```
enum E_FX
{
    E_FX_NONE = 0,
    E_FX_8s,
```

```

E_FX_1s_LR,
E_FX_1s_RL,
E_FX_GOLF_LR_SE,
E_FX_GOLF_RL_ES,
E_FX_GOLF_LR_ES,
E_FX_GOLF_RL_SE,
E_FX_SCROLL_UP,
E_FX_SCROLL_DOWN,
E_FX_SCROLL_LEFT,
E_FX_SCROLL_RIGHT,
E_FX_BLINK,
E_FX_PULSE
};

```

The enum *E_FX* has a value for each special effect supported by the display.
Note: Custom effects can be made using the API.

Structure SEG

```

typedef unsigned char SEGMENT;
struct structSeg
{
    const SEGMENT A;
    const SEGMENT B;
    const SEGMENT C;
    const SEGMENT D;
    const SEGMENT E;
    const SEGMENT F;
    const SEGMENT G;
    const SEGMENT DP;
    const SEGMENT ALL;
};
static struct structSeg SEG = {0x80,0x40,0x20,0x10,0x08,0x04,0x02,0x01,0xFF};

```

Each element of SEG relates to a segment of the display. The relation can be seen in the image below.

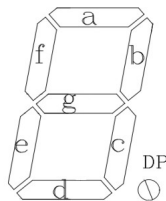


Fig.1 – Identification of each segment

Structure DIG

```

typedef unsigned char DIGIT;
struct structDig
{
    const DIGIT AUTO;
    const DIGIT D1;

```

```

const DIGIT D2;
const DIGIT D3;
const DIGIT D4;
const DIGIT D5;
const DIGIT D6;
const DIGIT D7;
const DIGIT D8;
const DIGIT ALL;
};
static struct structDig DIG = {0,0x80,0x40,0x20,0x10,0x08,0x04,0x02,0x01,0xFF};

```

Each element of DIG relates to a digit of the display, being D1 the leftmost digit and D8 the rightmost. As may be seen ahead, the AUTO is used to auto-detect which digits will be changed.

Structure DCHAR

This structure contains the 26 letters of the alphabet both lowercase and uppercase. It can be used to write text to the display.

Available helper classes:

Class CustomDisplay

This class is used to store custom data. This allows to write non numeric characters on the display as well as custom data. It can be used to make effects.

Class CustomDisplayWithIntensity

The same as CustomDisplay but it also stores information about the intensity of each segment of each the digit. These objects must be manipulated through the CreateCustomDisplayWithIntensity command which will be explained later in this document.

Class CustomFont

This classes defines a new font. A Font consists of different levels of intensity for the different segments of a digit.

3.4.2 Jackpot Display access API

All calls to the API are made through the singleton class instance: CUSBJackpot::getObject()
They all return a bool value indicating if the command was correctly sent to the USB display.

3.4.2.1 Write data on the display

The main feature is the one which allows to write a number on the display. There are several APIs to accomplish this.

```
bool SendNumber(double p_number, int p_decimalplaces = 2)
```

Parameter `p_number` is the number to show on the display and `p_decimalplaces` are the decimal places to appear. If `p_number` has more than `p_decimalplaces` the number is truncated otherwise the remaining decimal places are filled with zeros.

```
bool SendNumber(const char *p_number)
```

With this API `p_number` is a pointer to an array of chars. These chars can be numbers, decimal point and the minus "-" sign.

Custom Display

```
bool SendCustomDisplay(CustomDisplay* p_data,  
    const bool p_overwrite = true)
```

Use this command to write a custom data on the display. The parameter `p_data` is a pointer to a `CustomDisplay` class object which must have the data to be shown on the display. If the parameter `p_overwrite` is false then the `p_data` is added to the current data being shown on the display, if it's true only the `p_data` is shown on the display.

Custom Display With Intensity

To create a custom display where different segments can have different intensities use the following command:

```
bool CreateCustomDisplayWithIntensity(const SEGMENT p_segment,  
    const DIGIT p_digit,  
    const E_Intensity p_intensity,  
    CustomDisplayWithIntensity* p_myframe,  
    const bool p_overwrite = true )
```

The parameters `p_segment` and `p_digit` indicate which segments in which digits are to be shown and their intensity level is set with `p_intensity`. That information is stored in the `p_myframe` object. The `p_overwrite` indicates if this command should overwrite the information on the `p_myframe` or if should add it.

Note: This command does not write anything on the display.

To write a `CustomDisplayWithIntensity` object onto the display use the command:

```
bool SendCustomDisplayWithIntensity(CustomDisplayWithIntensity* p_myframe,  
    const bool p_overwrite = true)
```

where `p_myframe` is the object containing the data and `p_overwrite` indicates if this command should overwrite the information on the `p_myframe` or if should add it.

3.4.2.2 Special effects

This set of APIs allow the developer to animate the display.

```
bool SendFX(E_FX p_effect,  
            unsigned char p_delay,  
            bool p_sendWithNextScreen,  
            DIGIT p_digits = 0,  
            unsigned char p_repetitions = 0)
```

This command can be used to play a predefined FX listed in the E_FX enum. The usage of this command varies according to the E_FX.

The p_effect parameters selects the effect to use. The parameter p_delay sets the interval between frames of animation.

If p_sendWithNextScreen is true then the effect is buffered and it's applied with the next SendNumber command. This allows to use effects to change numbers.

The last two parameters only apply to E_FX_BLINK, E_FX_PULSE, E_FX_SCROLL_UP and E_FX_SCROLL_DOWN. The p_digits selects which digits will be affected by the FX and the last parameter, p_repetitions, set the number of repetitions of the effect .

```
bool SendVirusFX(int p_loopcount, int p_delay)
```

The virus FX shows random data on the display. The parameter p_loopcount sets the number of iterations of frames of the animation and the p_delay set the interval (ms) between frames.

```
bool SendMagmaFX(int p_loopcount, int p_delay)
```

The magma FX changes the intensity of the currently visible segments.

```
bool SendSnakeFX(const char* p_num)
```

The snake FX plays an animation around a number on the display. The p_num parameters sets the number to appear on the display.

Note: All the special effects APIs are blocking, meaning that the display will discard all information received while playing an animation.

3.4.2.3 Fonts

To set a new font on the display use the command

```
bool SetFont(CustomFont* p_font)
```

where the p_font is a CustomFont object previously created. To set a random font on the display use the command:

```
bool SetRandomFont()
```

3.4.2.4 Generic commands

```
bool Shift(const bool p_left, const unsigned char p_nplaces, const bool p_allowloop)
```

If `p_left` is true the display is shifted to the left otherwise is shifted to the right. The length of the shift is set by the `p_nplaces` parameters and the `p_allowloop` when set to true makes the data reappear on one side of the display when is popped off the other.

To center the data on the display use the command:

```
bool Center(void)
```

To change the intensity of the display:

```
bool SetIntensity(E_Intensity p_intensity,  
bool p_sendWithNextScreen = false)
```

The parameter `p_intensity` is an enum indicating the new level. If the change of intensity is only to be applied with the next change of data on the display, set the `p_sendWithNextScreen` parameter to true.

To reset the display to its original values (font, intensity and clear screen) use the command

```
bool Reset()
```

3.5 Demo program

The following program exemplifies the usage of the API.

```
#include "libjackdisplay.h"  
#include <iostream>  
#include <unistd.h> //for the usleep functions.  
  
using namespace USBJackpotDisplay;  
  
void SystemPause(void)  
{  
    printf("Press 'Enter' to continue: ... \n");  
    while ( getchar() != '\n');  
}  
  
int main()  
{  
    int l_i = 0;  
    CustomFont *l_font = new CustomFont(0xff,0xff,0xff);  
    CustomDisplay *l_jackpot = new  
    CustomDisplay(DCHAR.J,DCHAR.A,DCHAR.C,DCHAR.K,DCHAR.P,DCHAR.O,DCHAR.T1,DCHAR.T2); // "JACKPOT"  
    CustomDisplay *l_play = new CustomDisplay(0,0,0,0,DCHAR.P,DCHAR.L,DCHAR.A,DCHAR.Y); // "PLAY"  
  
    std::cout << "Starting Demo Application" << std::endl;  
    CUSBJackpot::getObject().SetIntensity(E_Intensity_2);  
    CUSBJackpot::getObject().SendNumber("8.");  
    SystemPause();  
    CUSBJackpot::getObject().SendNumber("8.8.8.8.8.8.8.");  
    SystemPause();  
    CUSBJackpot::getObject().SendNumber("11111111");  
    SystemPause();  
    CUSBJackpot::getObject().SendNumber("1234.56");  
    SystemPause();  
    CUSBJackpot::getObject().SendNumber("987654.32");  
    SystemPause();  
    CUSBJackpot::getObject().SendNumber(1234.56);
```

```

SystemPause();
CUSBJackpot::getObject().SendNumber(987654.32,1); // Truncates the number.
SystemPause();

while(l_i++ < 2000) // PT: Send 2000 commands each one with a different number.
{
    CUSBJackpot::getObject().SendNumber(l_i,0);
}
SystemPause();

//Plays all the GOLF FX with the word JACKTOP at different speeds
CUSBJackpot::getObject().SendCustomDisplay(l_jackpot,true);
CUSBJackpot::getObject().SendFX(E_FX_GOLF_RL_SE, 64, false, DIG.ALL);
SystemPause();
CUSBJackpot::getObject().SendFX(E_FX_GOLF_RL_ES, 30, false, DIG.ALL);
SystemPause();
CUSBJackpot::getObject().SendFX(E_FX_GOLF_LR_SE, 20, false, DIG.ALL);
SystemPause();
CUSBJackpot::getObject().SendFX(E_FX_GOLF_LR_ES, 10, false, DIG.ALL);
SystemPause();

//Scrolls the number two times.
CUSBJackpot::getObject().SendFX(E_FX_SCROLL_UP, 80, false, DIG.ALL ,2);
SystemPause();

//Scrolls the number. First D1, D3, D5, D7 and then D2, D4, D6, D8
CUSBJackpot::getObject().SendFX(E_FX_SCROLL_UP, 60, false, DIG.D1|DIG.D3|DIG.D5|DIG.D7,1);
CUSBJackpot::getObject().SendFX(E_FX_SCROLL_UP, 60, false, DIG.D2|DIG.D4|DIG.D6|DIG.D8,1);
CUSBJackpot::getObject().SendFX(E_FX_SCROLL_UP, 60, false, DIG.D1|DIG.D3|DIG.D5|DIG.D7,1);
CUSBJackpot::getObject().SendFX(E_FX_SCROLL_UP, 60, false, DIG.D2|DIG.D4|DIG.D6|DIG.D8,1);
SystemPause();

//Write "PLAY" on the display
CUSBJackpot::getObject().SendCustomDisplay(l_play);
CUSBJackpot::getObject().Center();
SystemPause();

//Scroll 5 positions to the left
CUSBJackpot::getObject().SendFX(E_FX_SCROLL_LEFT,50,false,DIG.AUTO,5);

//Write numbers with spaces
CUSBJackpot::getObject().SendNumber("60 60 60");
SystemPause();

//Performs a countdown (in seconds)
CUSBJackpot::getObject().CountDown(0,0,10);
SystemPause();

CUSBJackpot::getObject().Reset();

//Change number with effect
CUSBJackpot::getObject().SendNumber("112300.33");
CUSBJackpot::getObject().SendFX(E_FX_8s, 40, true, DIG.ALL);
CUSBJackpot::getObject().SendNumber("245020.51");
SystemPause();

//Blinks each digit
for(l_i=0;l_i<8;l_i++)
    CUSBJackpot::getObject().SendFX(E_FX_BLINK, 50, false, (0x80>>l_i), 5);
SystemPause();

//Blinks all digits at once
CUSBJackpot::getObject().SendFX(E_FX_BLINK, 70, false, DIG.ALL, 5); // PT: Pisca todos
SystemPause();

CUSBJackpot::getObject().SetFont(FONTEXAMPLE1);
SystemPause();

CUSBJackpot::getObject().SetRandomFont();
SystemPause();

CUSBJackpot::getObject().SetFont(l_font);
SystemPause();

```

```

// Apply intensity FX
CUSBJackpot::getObject().SendIntensityFX(4,true, 50);
SystemPause();

//Decreases intensity
CUSBJackpot::getObject().SetIntensity(E_Intensity_3);
SystemPause();
CUSBJackpot::getObject().SetIntensity(E_Intensity_2);
SystemPause();
CUSBJackpot::getObject().SetIntensity(E_Intensity_1);
SystemPause();

// Apply intensity FX
CUSBJackpot::getObject().SendIntensityFX(15,false,15);    // PT: efeito kit (normal)
SystemPause();

// Increase intensity FX
CUSBJackpot::getObject().SetIntensity(E_Intensity_2);
SystemPause();
CUSBJackpot::getObject().SetIntensity(E_Intensity_3);
SystemPause();
CUSBJackpot::getObject().SetIntensity(E_Intensity_4);
SystemPause();

// Apply intensity FX
CUSBJackpot::getObject().SendFX(E_FX_BLINK, 100, false, DIG.ALL, 10);
SystemPause();
CUSBJackpot::getObject().SetIntensity(E_Intensity_1);

//Change numbers with effects
CUSBJackpot::getObject().SendFX(E_FX_1s_LR,60,true,DIG.ALL);
CUSBJackpot::getObject().SendNumber("999999");
SystemPause();
CUSBJackpot::getObject().SendFX(E_FX_1s_RL,60,true,DIG.ALL);
CUSBJackpot::getObject().SendNumber("1000000");
SystemPause();

//Use the PULSE effect only on the changing digits
CUSBJackpot::getObject().SendNumber("9467.32");
SystemPause();
CUSBJackpot::getObject().SendFX(E_FX_PULSE, 50, true, DIG.AUTO, 10);
CUSBJackpot::getObject().SendNumber("9467.56");
SystemPause();

// Left shift without loop (expected result "7.56      ")
CUSBJackpot::getObject().Shift(true, 5, false);
SystemPause();

// Right shift with loop (expected result "56      7.")
CUSBJackpot::getObject().Shift(false, 7, true);
SystemPause();

CUSBJackpot::getObject().SendNumber("9467.56");
SystemPause();
CUSBJackpot::getObject().SendFX(E_FX_PULSE, 50, false, DIG.ALL, 4);

//Show the words "door" and "OPEN".
CUSBJackpot::getObject().SetIntensity(E_Intensity_4);
CustomDisplay *l_door = new CustomDisplay(0,0,DCHAR.d,DCHAR.o,DCHAR.o,DCHAR.r,0,0);
CustomDisplay *l_open = new CustomDisplay(0,0,DCHAR.O,DCHAR.P,DCHAR.E,DCHAR.N,0,0);
CUSBJackpot::getObject().SendCustomDisplay(l_door);
usleep(1000*1000);
CUSBJackpot::getObject().SendCustomDisplay(l_open);
usleep(1000*1000);
CUSBJackpot::getObject().SendCustomDisplay(l_door);
usleep(1000*1000);
CUSBJackpot::getObject().SendCustomDisplay(l_open);
usleep(1000*1000);

CUSBJackpot::getObject().SendVirusFX(40,200);
SystemPause();

CUSBJackpot::getObject().SendMagmaFX(50,100);

```

```
CUSBJackpot::getObject().SendFX(E_FX_GOLF_LR_ES, 80, true, DIG.ALL);  
CUSBJackpot::getObject().SendNumber(123);  
  
return 0;  
}
```

4 Hardware

4.1 Device Description

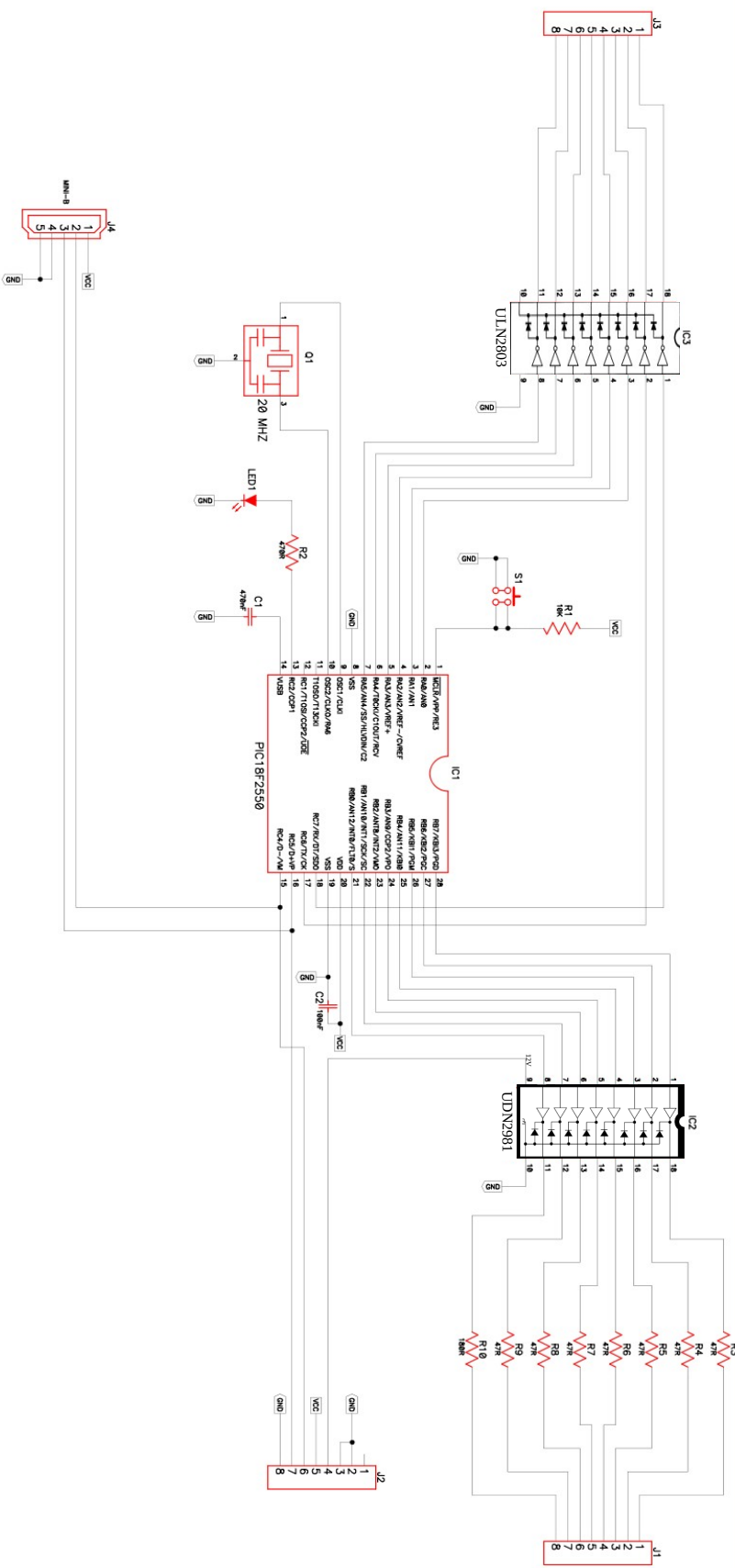
This device has:

- a reset button (S1) – reboots the device
- the J2 interface with the following contacts:
 - 2, 3 and 8 – ground
 - 4 – 12 Volts
 - 5 – Vdd (USB secondary interface)
 - 6 – D- (USB secondary interface)
 - 7 – D+ (USB secondary interface)
- mini USB interface (J4)
- display interface (8 contacts for the digits and 8 for the segments)

4.2 Technical Specifications

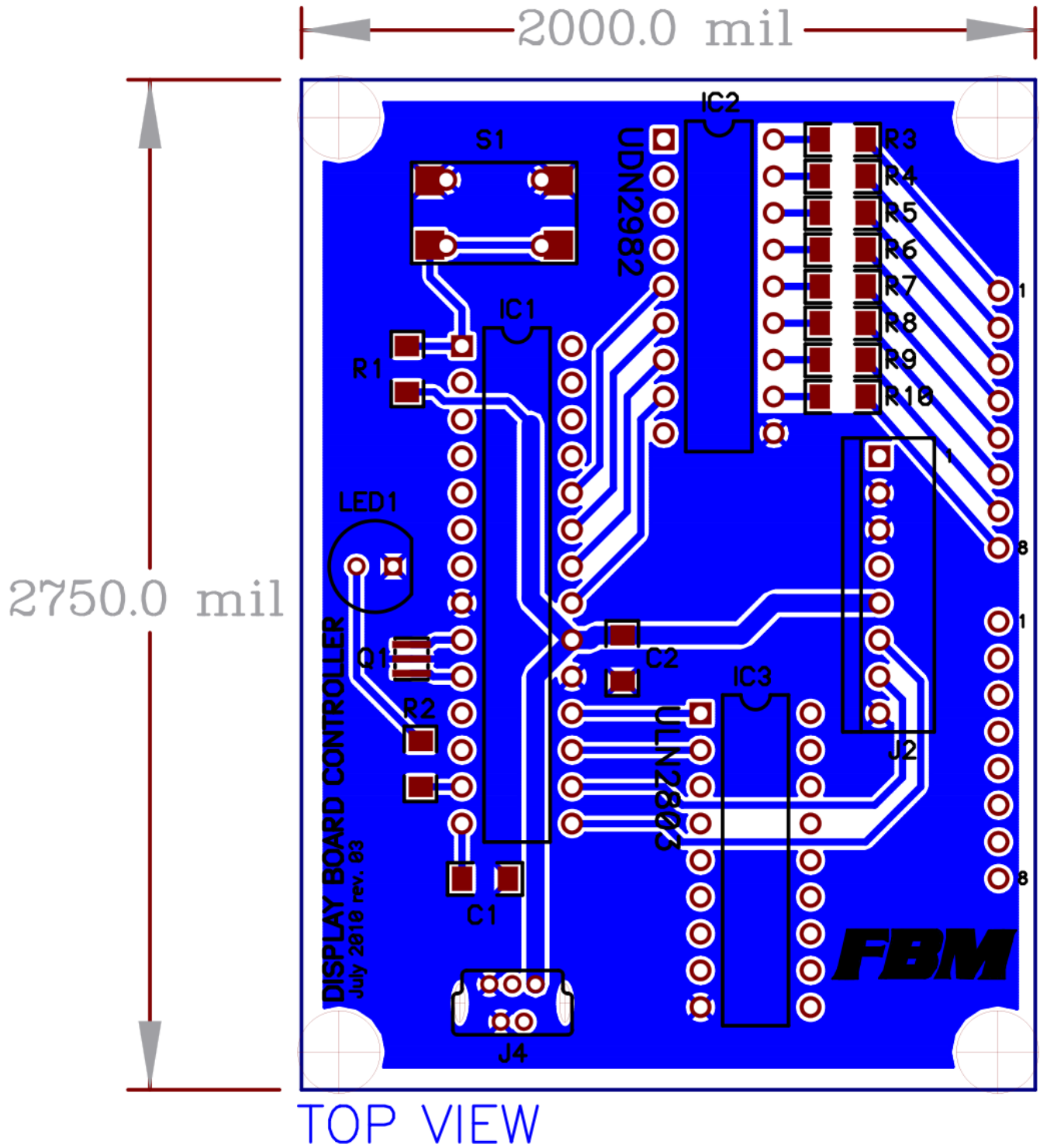
Interface	USB
Dimensions	7cm x 5cm x 2cm
Input Power	12V DC (and USB power)
Maximum Output Current (to Display)	500 mA
Operating Temperature	–20 to 80 °C

4.3 Electrical Schematic

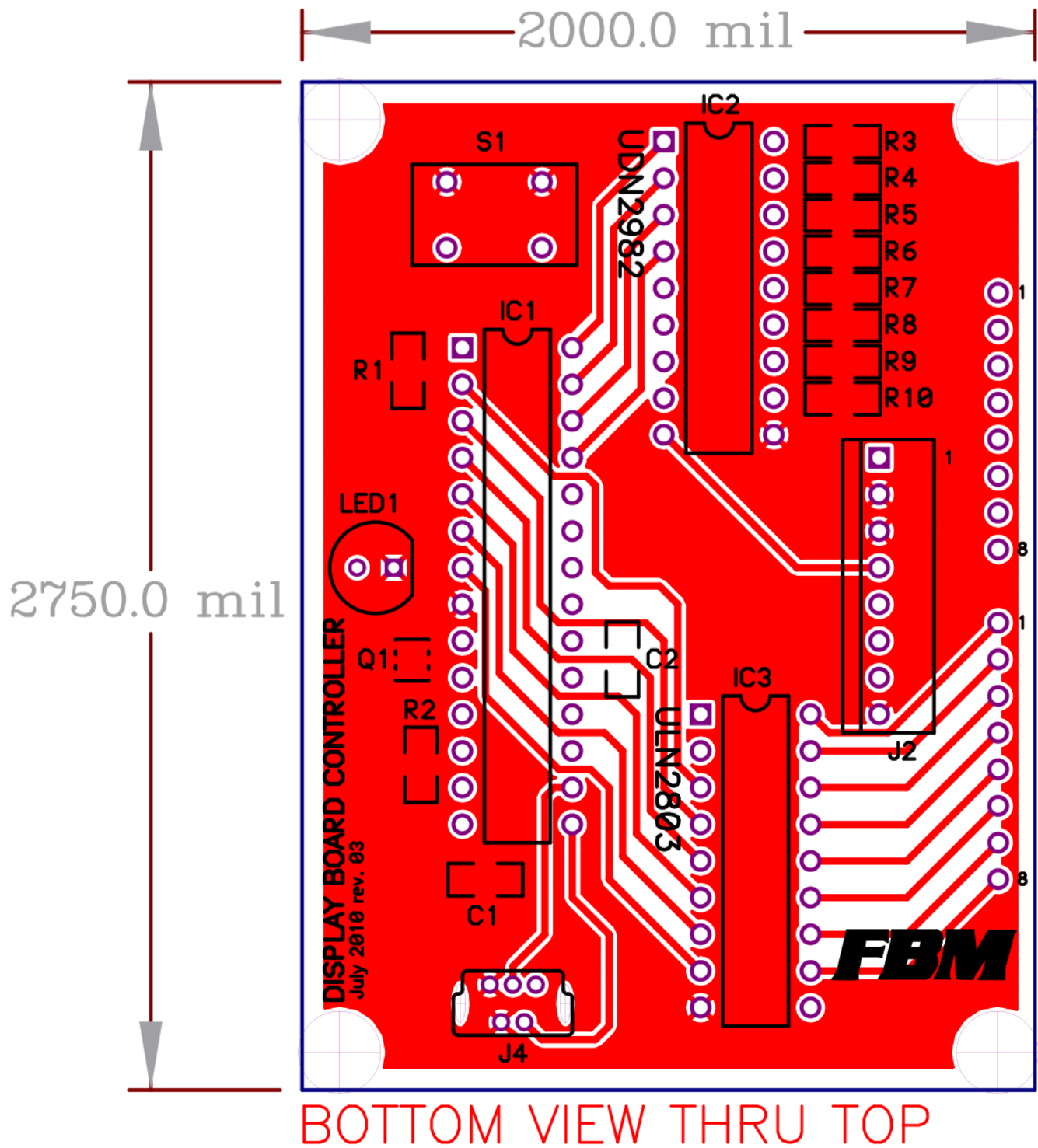


4.4 Board Layout

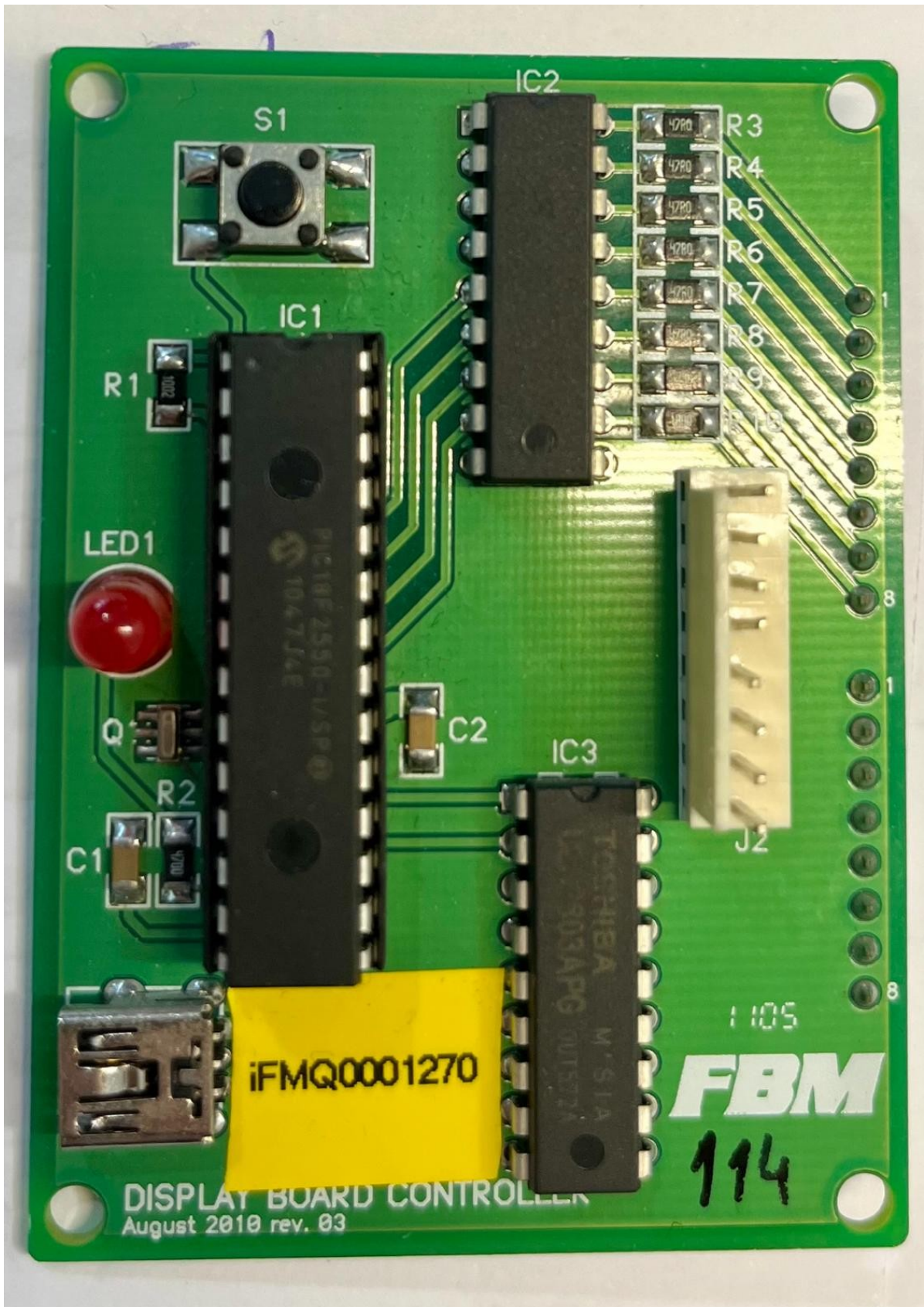
4.4.1 Top View

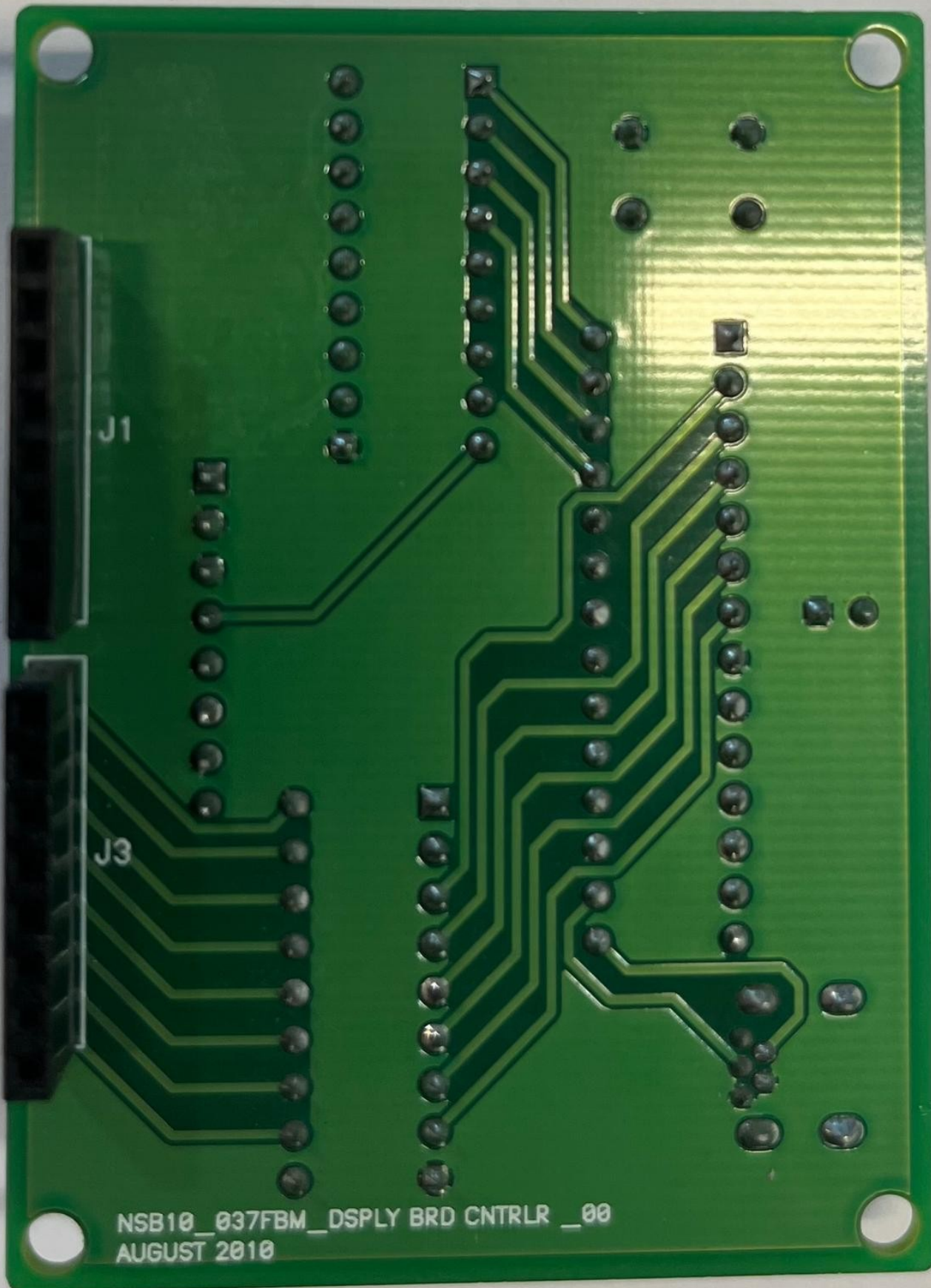


4.4.2 Bottom View



4.5 Board Photos





NSB10_037FBM_DSPLY BRD CNTRLR _00
AUGUST 2010

5 FAQ

- 1) I've connected the device and the LED lighted up. Why doesn't displays anything?

Verify the display's 12V power on the device. This must be attached to contact 4 of interface J2 (page 16).

Verify if the display has been attached with the USB device in the correct way (overlapping the device).

- 2) I've connected the device and the LED didn't lighted up.

Verify if the USB cable is correctly attached.